Ultrasonic Distance Visualization and Alert System Final Report

Lucas Yang

Introduction

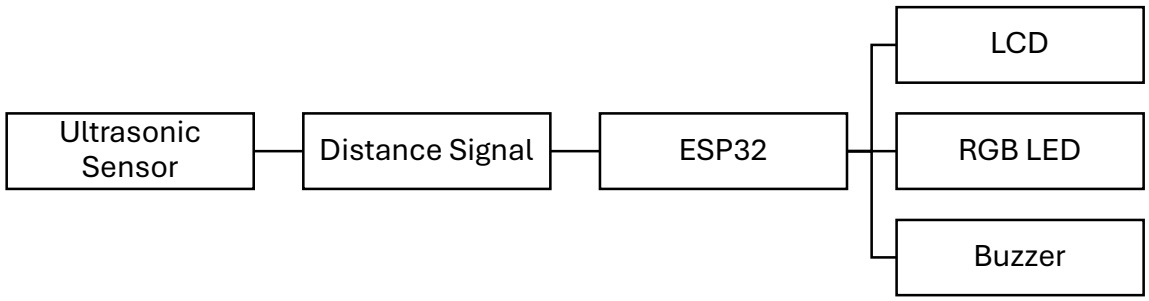
In modern embedded systems, real-time sensing and intuitive user feedback are critical for creating user-friendly devices. Distance measurement, in particular, has a vital role in applications including: obstacle avoidance, parking assistance, and positioning.

This project presents the design and implementation of an Ultrasonic Distance Visualization and Alert System by using the ESP32 MCU. The device utilizes the ultrasonic sensor to measure the distance from itself to the object nearby continuously. The data measured will be processed and conveyed to the user via multiple output modalities, including a liquid crystal display for numerical readings and a RGB LED for more intuitive feedback. The device will also include an audible alert to enhance situational awareness when objects are within critical distance.

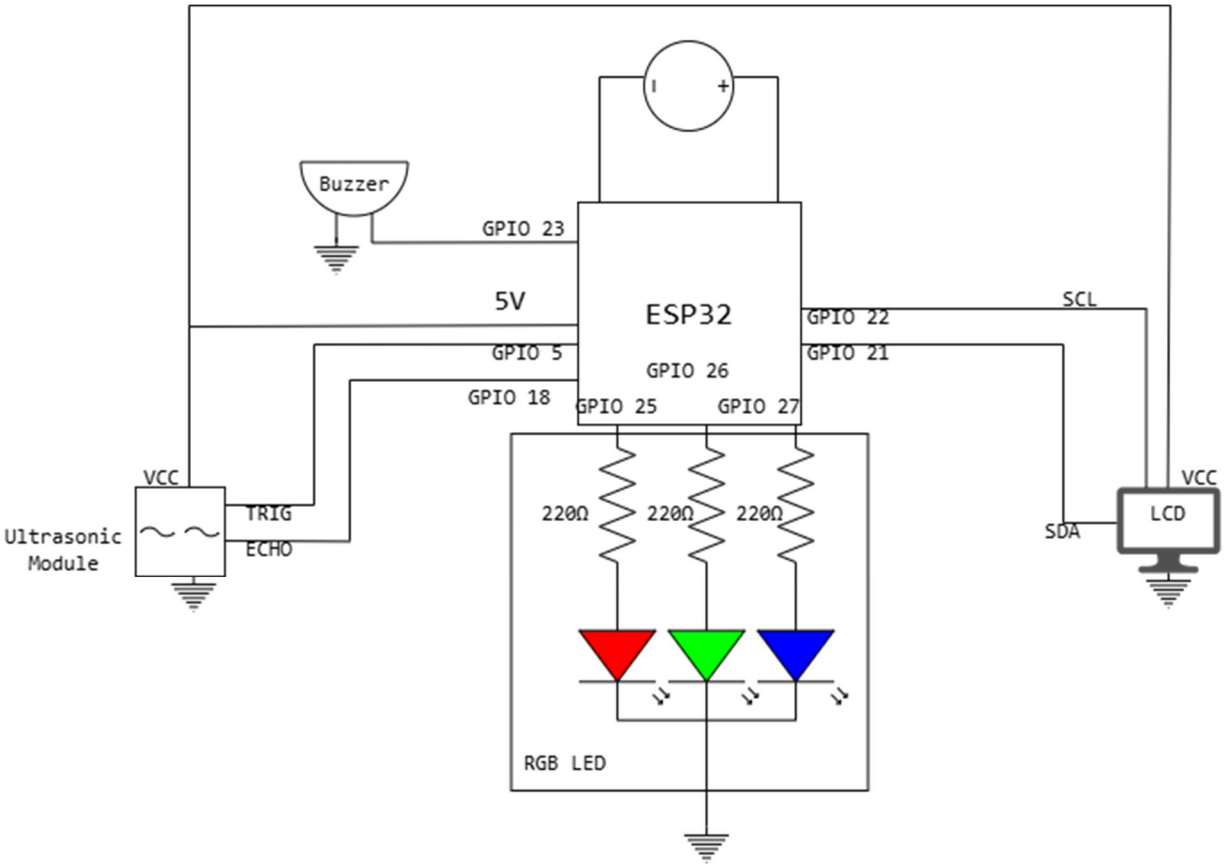
The motivation behind this project is to explore the integration of sensing, processing, and multi-modal feedback within a single embedded system. By combining quantitative display with color-based visualization and auditory alerts, the system aims to improve user interaction and demonstrate fundamental principles of real-time embedded system design.

Design

1. Components:
    a. ESP32 MCU
    b. Ultrasonic Module
    c. I2C LCD
    d. RGB LED
    e. 220Ω Resistors ×3
    f. Buzzer
    g. Breadboard
    h. Jumper Wire (Depends on actual circuit)
    i. USB-C Wire
2. Block Diagram (Data goes left to right)



3. Schematic



4. Software Plan

Initialize LCD

Initialize Ultrasonic Sensor

Initialize RGB LED

Initialize Buzzer

Set distance_thresholds:

safe_distance = 30 cm

warning_distance = 15 cm

Loop forever:

Trigger ultrasonic sensor

Read echo time

Calculate distance

Display distance on LCD

```
if distance > safe_distance:
    Set RGB LED color to BLUE
    Turn off Buzzer
    Display "Safe" on LCD

else if distance > warning_distance:
    Set RGB LED color to GREEN
    Turn on Buzzer with slow beep
    Display "Caution" on LCD

else:
    Set RGB LED color to RED
    Turn on Buzzer with fast beep
    Display "Danger" on LCD

Wait short delay 100ms
```

Complexity

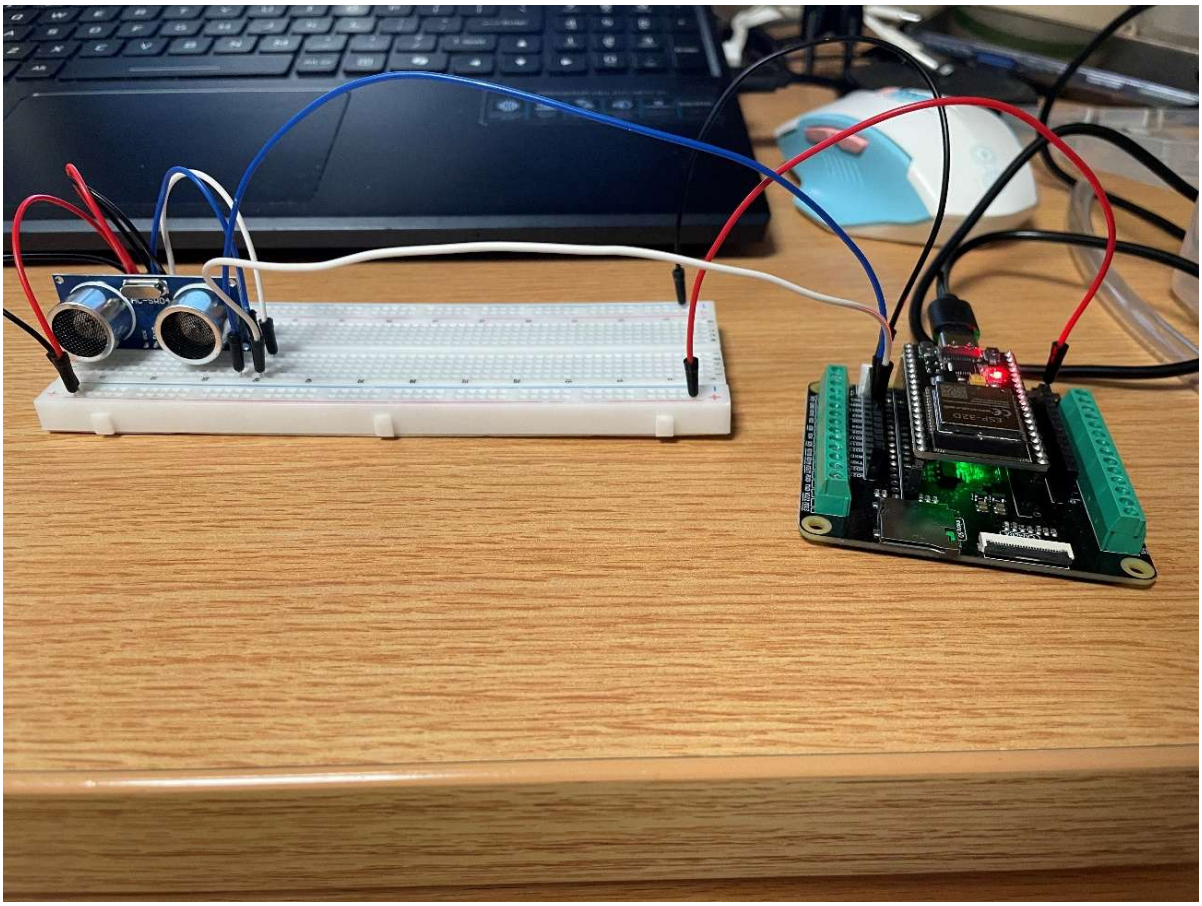
1. Input: The time gap between two signals sent by the Ultrasonic Module
2. Output: Color by RGB LED, sound by buzzer and numerical readings on LCD
3. New component: RGB LED, buzzer, LCD
4. Complexity Score:4(Ultrasonic Module) + 4(LCD) + 3(Buzzer) + 2(RGB LED) + 0(ESP32 + Breadboard + Resistors + Jumper Wires) = 13

References

1. HC-SR04 Ultrasonic Sensor Datasheet.
2. ESP32 Technical Reference Manual, Espressif Systems.
3. I2C LCD Interface Documentation, Arduino.
4. Online example codes and tutorials for ESP32 sensor interfacing were consulted during development.

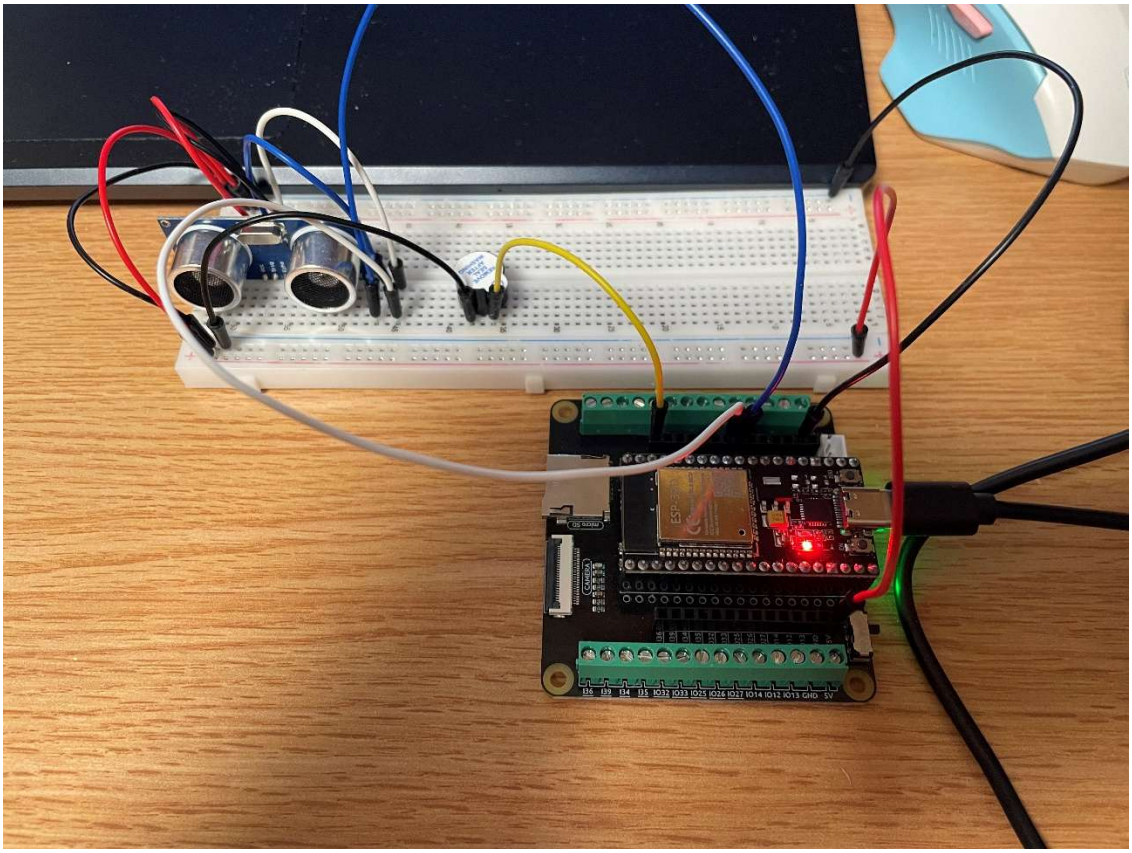
Building Process:

1. Process:
   a. Ultrasonic Module:



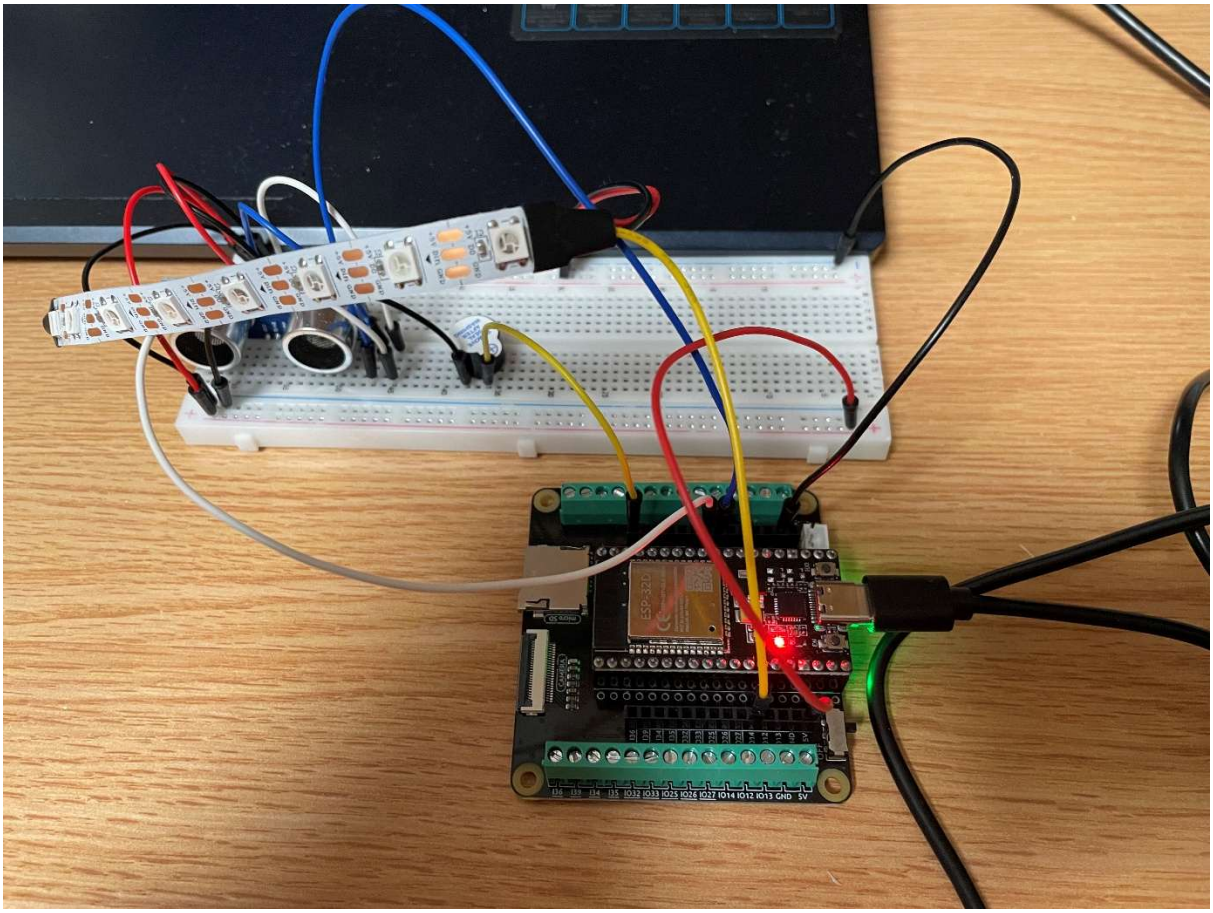
Ultrasonic Module was attached to the breadboard with ECHO pin to IO18, TRIG pin to IO5, VCC to power bus and GND to ground bus.

b. Buzzer:



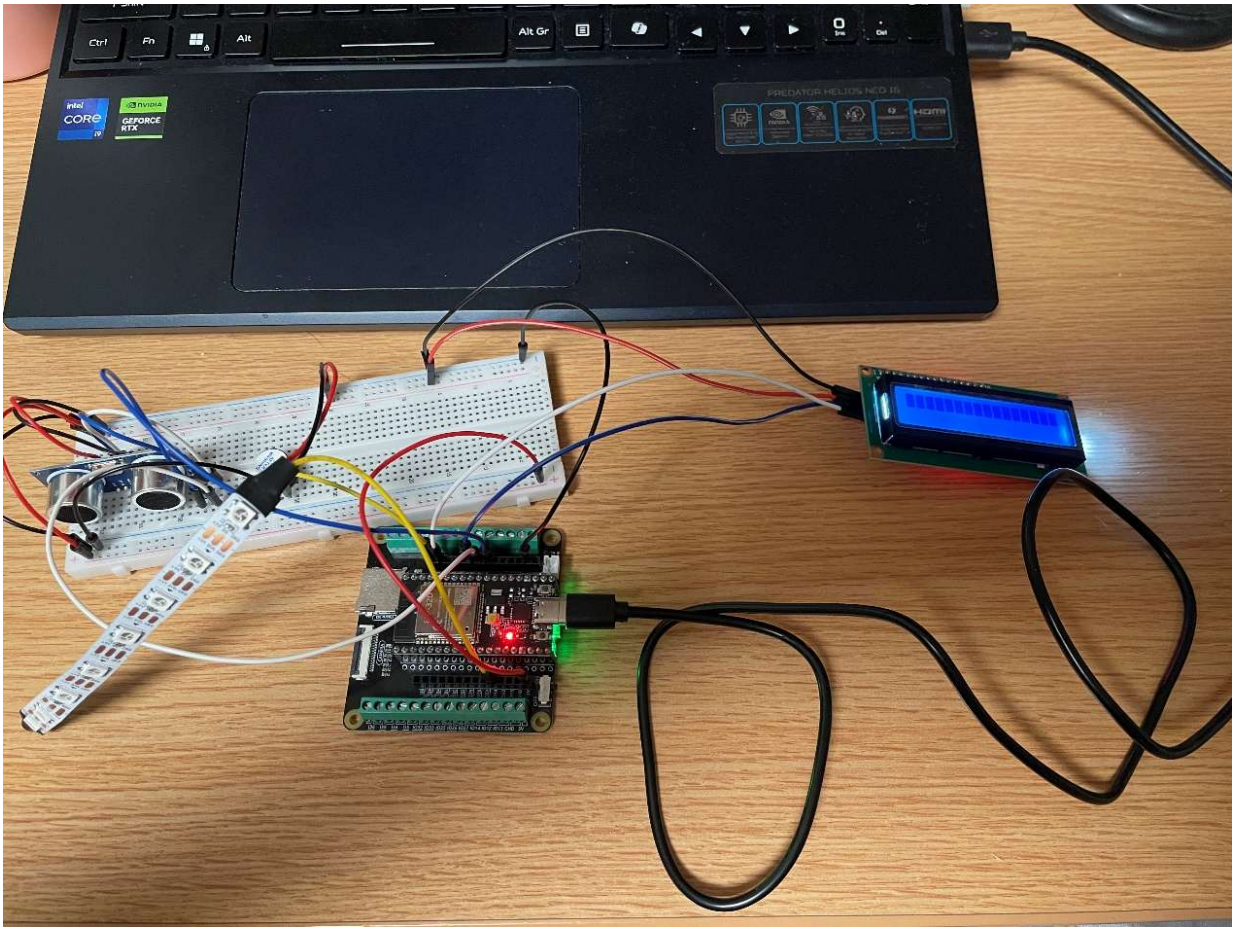
Buzzer was attached to the breadboard with positive to IO23, negative to ground bus.

c. LED Strip:



LED Strip was attached to the breadboard via power and ground line, with control line connected to IO32.

d. LCD:



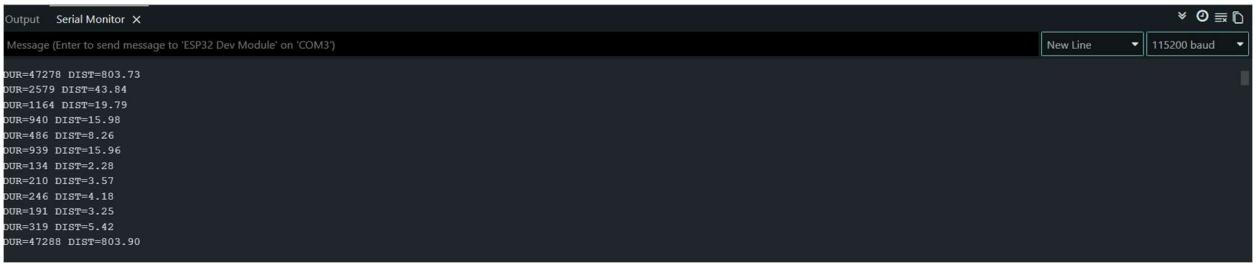
LCD was attached to breadboard with SDA to IO19, SCL to IO22 and VCC to power bus and GND to ground bus.

2. Tests:

a. Sensor Testing (Ultrasonic Module):

The first step in the build process was to test the ultrasonic sensor module independently. The sensor was connected to the ESP32, and a simple program was used to output distance readings to the serial monitor.

This step verified that the sensor was powered correctly and able to produce valid distance measurements before integrating other components. The screenshot below shows the result of the test, duration is the time between sending and receiving the signal, the distance shows the distance to the target.



b. Actuator Testing (WS2812 RGB Strip, Active Buzzer, I2C LCD 1602):

i. LED Test: Initially, the idea was to use a standard 4 pin RGB LED. However, generating smooth color transitions with RGB LED requires complex wiring and reduces power efficiency. It also needs PWM control for each color channel. To simplify the hardware and improve flexibility and visuality, the RGB LED was replaced with RGB Strip. The LED strip was then tested independently to verify that it could display various colors correctly and continuously respond to software control. This change reduced circuit complexity and allowed smoother color transitions to be implemented entirely in software. LED.mp4

ii. Buzzer Test: The buzzer was tested by generating a constant tone using the ESP32. This verified that the buzzer was functioning correctly and responding to GPIO signals. buzzer.mp4

iii. LCD Test: The LCD display was tested using a basic "Hello World" program to confirm proper I2C communication and display functionality. LCD.mp4

3. 3. Integration Testing: After verifying individual components, the system was gradually integrated. The ultrasonic sensor was first combined with the LED strip, followed by the buzzer and LCD display. At each stage, functionality was tested to ensure that adding new components did not interfere with existing behavior. More detailed information about the integration challenges and solutions will be discussed in the following section.

4. Trouble Shooting: During the integration process, several issues were encountered. Each of these issues requires systematic debugging and thorough testing to identify the deeper causes.

a. Problem I: No Echo Signal
One of the first issues encountered was that the ultrasonic module consistently returns 0 for duration value, which means no echo signal was detected or sent to the MCU. After checking wiring carefully, finding that the jumping wire for the echo signal was falsely connected. The problem was quickly fixed, and valid readings began to appear.

b. Problem 2: Echo Signals Exists but Not Read by MCU
In some cases, the sensor appeared to generate signals, but MCU wasn’t detecting them correctly. Observed distance seldom returned valid results, but over 95% of the data were 0, GPIO readings remained in LOW. The method used to debug be to try to visualize the issue by plugging in an LED with a resistor directly in series on the echo signal line. It turns out that the LED blinks frequently, confirmed that signal is successfully generated, so that rules out the sensor itself to be the cause of the problem. However,

after re-wiring the circus to place the LED after the protector resistor, it didn’t blink for about 1000 readings, locked down the problem to the certain area. It turned out to be the reason for bad connections between bread board and the resistor, which was also quickly fixed.

c. Problem 3: Unstable Readings During Integration
After integrating all components, the sensor reading became unstable and inconsistent. From outside, the actuators did respond to input signals, but each actuator has shown different problems. RGB Strip only shows red color, which in the program indicates the closest distance, buzzer buzzing at the highest frequency and LCD either showing unreadable characters or just white blocks only. Situations discussed above happened at any distance. From inside, the value of the distance stays 0 or -1.
From the individual test, actuators work fine, and the signal should be transmitted properly, in this case, the only cause of the disorder can only come from the software. By doing individual tests on each function, the instability was likely caused by timing interference from other components, especially LCD updates and LED strip control. The way to solve this problem was to redesign to separate tasks and control timing using millis() function. The instability was likely caused by timing interference from other components, especially LCD updates and LED strip control.

d. Outcome: After resolving these issues, the system achieved stable and reliable performance. The ultrasonic sensor produced consistent distance readings, and all output components responded correctly without interfering with each other.

Functionality: Final Result.mp4

In the demonstration video, a small object (a box) is gradually moved from a farther distance toward the sensor. As the object approaches, the system responds dynamically through multiple outputs.

The LED strip changes color based on distance, transitioning from blue at farther distances to green and then red as the object gets closer. The buzzer also adjusts its behavior, with the beeping frequency increasing as the distance decreases. At the same time, the LCD display updates to show the current distance and corresponding system status.

Due to limitations in video capture, the color transitions of the LED strip may not appear as clearly as in real life; however, the changes are more noticeable when observed directly

Complexity:

1. Complexity Score: Buzzer(3) + I2C LCD(4) + WS2812 RGB Strip(4) + Ultrasonic Module(4) + Resistor, ESP32, Breadboard, Jumper Wires(0) = 15

2. Input: The system uses an ultrasonic sensor as the primary input device. It continuously measures the distance between the sensor and nearby objects, providing real-time data to the ESP32.

3. Output: The project includes multiple output components that respond dynamically to the measured distance:

    a. LED Strip :Displays a continuous color gradient (red to green to blue) based on distance
    b. Buzzer: Provides auditory feedback with increasing beep frequency as the object approaches
    c. LCD Display: Shows real-time distance values and system status (SAFE, WARNING, DANGER)

4. New Components: In addition to basic components, the project incorporates more advanced modules:
    a. An addressable LED strip, which integrates control circuitry and allows software-based color control

    b. An I2C LCD display, which reduces wiring complexity compared to parallel interfaces

5. Overall, the system includes one sensor input and multiple coordinated outputs, exceeding the minimum project requirements. The integration of multiple modules and real-time feedback mechanisms demonstrates a higher level of system complexity.

Reflection and Future Improvements:

Overall, the project successfully achieved its main objective of building a distance sensing system with multiple forms of feedback, including visual, auditory, and display outputs. The system was able to measure distance reliably and respond in real time using the LED strip, buzzer, and LCD.

One of the key strengths of this project was the successful integration of multiple components into a cohesive system. The LED strip provided smooth color transitions, the buzzer offered intuitive audio feedback, and the LCD displayed clear and readable status information. Additionally, restructuring the program using timing control with millis() significantly improved system stability and reduced interference between components.

During development, several challenges were encountered, particularly related to sensor reliability and timing conflicts during integration. Issues such as missing echo signals, unstable readings, and interference from other components required careful debugging and iterative testing. These challenges highlighted the importance of both correct hardware connections and proper software timing design in embedded systems.

Several improvements could be made to further enhance the system. Additional user input, such as buttons, could be incorporated to allow configuration of warning and danger thresholds, similar to a digital alarm clock, making the system more flexible and interactive. In terms of sensing accuracy, the ultrasonic sensor could be replaced or complemented with an optical/photoelectric sensor to achieve higher precision and faster response times. Using both sensors together could further improve reliability by reducing errors caused by echo scattering or environmental conditions, effectively applying a simple form of sensor fusion.

Further system enhancements could include implementing more advanced filtering algorithms, improving visual effects on the LED strip, or upgrading to a graphical display for better user experience.

This project provided valuable experience in both hardware integration and software design. It reinforced key aspects of an engineering mindset, such as breaking down complex problems into manageable components, testing each part independently, and iteratively refining the system based on observed behavior. Rather than expecting the system to work perfectly on the first attempt, the development process emphasized troubleshooting, validation, and continuous improvement.

In conclution, this project demonstrates how an initial concept can be progressively developed into a functional and potentially practical system, highlighting the feasibility of transforming a simple idea into a more complete and application-oriented solution.

Appendix A: Project Code

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Adafruit_NeoPixel.h>

//PIN CONFIGURATION
const int trigPin = 5;
const int echoPin = 18;
const int buzzerPin = 23;
const int ledPin = 32;
const int ledNum = 8;

//LCD & LED SETUP
LiquidCrystal_I2C lcd(0x27, 16, 2);
Adafruit_NeoPixel strip(ledNum, ledPin, NEO_GRB + NEO_KHZ800);

//TIMING VARIABLES
volatile long startTime = 0;
volatile long duration = 0;
volatile bool done = false;

float distance = -1;

//INTERRUPT SERVICE ROUTINE (ECHO SIGNAL)
void IRAM_ATTR echoISR() {

  // Rising edge: start timing
  if (digitalRead(echoPin) == HIGH) {
```

```
    startTime = micros();
  }
  // Falling edge: calculate pulse duration
  else {
    duration = micros() - startTime;
    done = true;
  }
}

//TRIGGER ULTRASONIC SENSOR
void triggerSensor() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(3);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
}

//CONVERT DURATION TO DISTANCE
float computeDistance(long d) {
  if (d <= 0) return -1;
  return d * 0.034 / 2;
}

//LED COLOR UPDATE (DISTANCE → COLOR)
void updateLED(float d) {

  if (d < 0) {
    strip.clear();
    strip.show();
```

```
  return;
}

int r = 0, g = 0, b = 0;

// Below 15 cm: RED → GREEN
if (d < 15) {

  float t = d / 15.0;   // normalized value (0 → 1)

  r = (int)(255 * (1 - t));
  g = (int)(255 * t);
  b = 0;
}

// 15–30 cm: GREEN → BLUE
else if (d <= 30) {

  float t = (d - 15) / 15.0;  // normalized value (0 → 1)

  r = 0;
  g = (int)(255 * (1 - t));
  b = (int)(255 * t);
}

//Above 30 cm: BLUE
else {
  r = 0;
  g = 0;
  b = 255;
```

```
  }

  //APPLY COLOR TO ALL LEDs
  for (int i = 0; i < ledNum; i++) {
    strip.setPixelColor(i, strip.Color(r, g, b));
  }

  strip.show();
}

//BUZZER CONTROL
unsigned long lastBeep = 0;

void updateBuzzer(float d) {

  // Disable buzzer if out of range
  if (d < 0 || d > 30) {
    noTone(buzzerPin);
    return;
  }

  //Map distance to beep interval
  int interval = map(d, 30, 1, 800, 50);
  // 30 cm → slow (800 ms)
  // 1 cm  → fast (50 ms)

  unsigned long now = millis();

  if (now - lastBeep >= interval) {
    lastBeep = now;
```

```
    tone(buzzerPin, 2000, 30); // short beep pulse
  }
}

//LCD DISPLAY
unsigned long lastLCD = 0;

void updateLCD(float d) {

  //Line 1: Distance
  lcd.setCursor(0, 0);
  lcd.print("Dist:           ");
  lcd.setCursor(6, 0);

  if (d < 0) {
    lcd.print("ERR      ");
  } else {
    lcd.print(d, 1);
    lcd.print(" cm   ");
  }

  //Line 2: Status
  lcd.setCursor(0, 1);

  if (d < 0) {
    lcd.print("NO SIGNAL       ");
  }
  else if (d > 30) {
    lcd.print("SAFE            ");
```

```
  }
  else if (d > 15) {
    lcd.print("WARNING         ");
  }
  else {
    lcd.print("DANGER          ");
  }
}

//SETUP
void setup() {

  Serial.begin(115200);

  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(buzzerPin, OUTPUT);

  // Attach interrupt to echo pin
  attachInterrupt(digitalPinToInterrupt(echoPin), echoISR, CHANGE);

  lcd.init();
  lcd.backlight();

  strip.begin();
  strip.show();

  lcd.print("STABLE MODE");
  delay(1000);
  lcd.clear();
```

```
}

//MAIN LOOP
void loop() {

  // Trigger sensor
  triggerSensor();

  // Wait for echo signal (handled by ISR)
  delay(50);

  // Safely read shared variables
  noInterrupts();
  long d = duration;
  bool ok = done;
  done = false;
  interrupts();

  // Compute distance if valid signal received
  if (ok) {
    distance = computeDistance(d);
  } else {
    distance = -1;
  }

  // Debug output
  Serial.print("DUR=");
  Serial.print(d);
  Serial.print(" DIST=");
  Serial.println(distance);
```

```
  // Update outputs
  updateLCD(distance);
  updateLED(distance);
  updateBuzzer(distance);

  delay(100);
}
```

Appendix B: References

HC-SR04 Ultrasonic Sensor Datasheet.
Arduino Documentation. https://www.arduino.cc
ESP32 Technical Reference Manual.
Adafruit NeoPixel Library Documentation.
LiquidCrystal I2C Library Documentation.